gracefoReading

GRACE-FO Level-1B ASCII data reading toolbox

User Guide and API Reference

Version 1.0.2 — 2026-09-09

Reads any GRACE-FO Level-1B RL04 ASCII product (ACT1B/ACC1B, SCA1B, KBR1B, LRI1B, GNV1B, THR1B, MAS1B, ...) into a common time/obs/aux layout, driven entirely by each file's own YAML header. Supports split microsecond time tags, bit-string quality flags, prod_flag-controlled variable-length records, gzip input, and strict row-integrity checking. Validated against real PO.DAAC RL04 files of 2020-01-01. Includes batch tools (file discovery, gap analysis, content-hashed parse caching), exact int64 GPS-nanosecond time output, selective quality-bit filtering, GROOPS instrument export/import, and experimental classic GRACE (2002-2017) support.

Developed by Matthias Weigelt with the help of Anthropic Claude Opus 5, 2026-09-09.

License: MIT. Requires MATLAB R2020b or later.

Sections 6 and 7 are generated from the MATLAB help blocks, so this guide and the in-product help cannot disagree.

1 Overview

gracefoReading reads GRACE-FO Level-1B ASCII products into MATLAB and provides the surrounding tooling needed for real processing work: archive search, gap analysis, quality-flag decoding, content-hashed caching and GROOPS interoperability.

1.1 What the products look like

All GRACE-FO Level-1B ASCII products share one structure. A YAML header, terminated by the line "# End of YAML header", declares every data column in order under its variables key, together with units, comments and flag meanings. The records follow as whitespace-delimited lines.

Because the header is self-describing, readL1b needs no per-product column tables: it reads ACT1B/ACC1B, SCA1B, KBR1B, LRI1B, GNV1B, THR1B and MAS1B with the same code path, and a future product with the same header dialect will work without changes.

1.2 Details that decide correctness

Four properties of the real PO.DAAC RL04 files matter enough to state explicitly, because a reader that assumes otherwise produces plausible but wrong results:

- Split time tags: time_intg carries whole seconds and time_frac the fraction, but in THR1B and MAS1B the fraction is declared in MICROSECONDS. Assuming seconds shifts every epoch.
- Time system: GPS seconds past 2000-01-01 12:00:00, continuous, without leap seconds. Converting via a naive UTC path introduces an 18 second error.
- Flag columns: qualflg and prod_flag are 8-character bit STRINGS, not numbers. Bit 0 is the rightmost character. Reading them as numbers destroys the bit pattern.
- Variable-length records: in MAS1B the prod_flag decides which columns are present in each line, so a line has fewer tokens than there are declared variables. Absent values are returned as NaN.

1.3 Design principles

- Header-driven: metadata comes from the file, never from a hard-coded table, so the reader follows the products across releases.
- Fail loudly: a row-integrity violation names the exact line rather than silently producing a shorter array.
- Validated against real data: the acceptance suite reads genuine PO.DAAC files and compares against an independent Python reference implementation.
- Provenance: everything the reader learned about a file is returned in aux, and cache keys are SHA-256 digests of the parse inputs.

1.4 Toolbox layout

Path	Contents
`+gracefoReading/`	namespace package: public API and +internal helpers
`html/`	generated HTML reference, registered with the doc browser
`doc/`	this guide
`docsrc/`	documentation generators and the help-text audit
`tests/`	matlab.unittest suites, real data and Python reference
`examples/`	demoGracefoReading.m, an executable tour
`ci/`	runCI.m, the continuous-integration entry point
`buildfile.m`	buildtool tasks: test, audit, docs, check, guide, release

2 Installation

2.1 Requirements

- MATLAB R2020b or later (the toolbox uses arguments blocks).
- No additional MathWorks toolboxes are required.
- A JVM is used for in-memory gzip decompression and SHA-256 cache keys; under -nojvm both fall back gracefully.
- Python 3.9+ with reportlab only if you want to regenerate the documentation yourself.

2.2 Install

Unpack the archive anywhere you like, for example C:\Users\<you>\Documents\MATLAB\gracefoReading or ~/matlab/gracefoReading, then add it to the path:

```
cd <toolboxRoot>
setupGraceFOReading
```

setupGraceFOReading adds the toolbox root, which contains the +gracefoReading package folder, and registers the HTML documentation with the doc browser. The tests folder is deliberately not added to the path, so that fixture names can never shadow real functions.

For a permanent installation put the addpath call into your startup.m:

```
addpath('C:\Users\<you>\Documents\MATLAB\gracefoReading');
```

2.3 Verify

```
addpath('ci');
results = runCI();        % all suites; reports in artifacts/
disp(table(results));
```

A clean installation reports all tests passed. The suites are self-contained: synthetic fixtures are written to a temporary folder and removed afterwards, and the bundled real files in tests/realdata/ provide the acceptance checks.

2.4 Uninstall

Remove the folder from the path and delete it. Nothing is installed elsewhere; caches, if you used them, live in the directory you named with 'CacheDir'.

3 Build, test and continuous integration

Every repeatable action has one entry point, so that a release never depends on remembering a sequence of steps.

3.1 Build tasks

Command	Action
`buildtool test`	run every suite through ci/runCI.m; JUnit and Cobertura reports in artifacts/
`buildtool audit`	check all help blocks against the documentation standard (docsrc/auditHelp.py)
`buildtool docs`	regenerate html/ and this guide from the help blocks
`buildtool check`	run the documentation-consistency suite only
`buildtool guide`	regenerate this PDF alone
`buildtool release`	audit, test, docs and check, then package a versioned zip

buildtool requires R2022b or later. On earlier releases call ci/runCI.m and docsrc/generate_docs.py directly; the actions are identical.

3.2 Running the tests

```
addpath('ci');
results = runCI();                % interactive
results = runCI('Coverage', false);% faster, no instrumentation

% batch mode, exactly what CI runs:
%   matlab -batch "addpath('ci'); runCI('Exit', true)"
```

'Exit' matters: MATLAB in -batch mode returns exit status 0 even when tests fail, so without it a CI job stays green over a red test suite. Do not use it interactively, since it closes MATLAB.

3.3 Test suites

Suite	Scope
`testReadL1b`	reader core against synthetic fixtures: headers, time tags, flags, prod_flag, cache, classic path
`testRealData`	acceptance against bundled real PO.DAAC files, expectations from the Python reference
`testBatchTools`	findGaps, findL1bFiles, readGroopsInstrument, multi-file monotonicity
`testUtilities`	flag decoding, quaternion norm, GROOPS export, version metadata
`testDocConsistency`	code, documentation and metadata agree; help blocks meet the standard

3.4 Continuous integration

The repository ships .github/workflows/ci.yml, which runs the suites on Linux and Windows and audits the documentation. Both platforms are tested deliberately: the two defects that survived release testing in this toolbox were platform-specific, namely a Java gzip read that silently returned zeros and CRLF line endings from fopen 'wt' on Windows.

```
matlab -batch "addpath('ci'); runCI('Exit', true)"
```

Installation of runners, licensing on self-hosted machines, and the equivalent GitLab and Jenkins snippets are documented in docs/CI_SETUP.md.

3.5 Documentation workflow

The MATLAB help blocks are the single source of truth. docsrc/generate_docs.py parses them and writes both the HTML reference and this guide; docsrc/auditHelp.py enforces the structure that makes the parsing possible. Never edit html/ by hand: edit the help text and regenerate.

```
python3 docsrc/auditHelp.py      # structure check
python3 docsrc/generate_docs.py  # html/ + doc/ regeneration
```

4 Step-by-step guides

4.1 Read one product file

Start from a downloaded PO.DAAC file. The reader needs nothing but the path: column names, units and flag meanings come from the file.

```
[t, obs, aux] = gracefoReading.readL1b('ACT1B_2020-01-01_C_04.txt');

disp(aux.product);            % 'ACT1B'
disp(aux.obsNames');          % what each obs column is
disp(aux.numRecordsActual);   % records parsed

ix = strcmp(aux.obsNames, 'lin_accl_x');
plot(t(:,2)/3600, obs(:, ix));
xlabel('hours of day'); ylabel('linear acceleration x [m/s^2]');
```

- t(:,1) is the integer MJD, t(:,2) the seconds of day.
- Gzipped files work identically: pass the .gz path.
- aux.header keeps the raw header for provenance.

4.2 Select a time window and a time format

Restrict to an interval and choose the representation that fits the next processing step.

```
% one hour, as datetime
[t, obs] = gracefoReading.readL1b(f, 'TimeFormat', 'datetime', ...
    'TimeRange', [datetime(2020,1,1,0,0,0), datetime(2020,1,1,1,0,0)]);

% exact int64 GPS nanoseconds for a nanosecond pipeline
tns = gracefoReading.readL1b(f, 'TimeFormat', 'gpsns');
fprintf('%d ns, class %s\n', tns(1), class(tns));
```

- "mjd" (default) is the double pair, convenient for plotting.
- "gpsns" is exact integer arithmetic; a double cannot hold 2020-era GPS nanoseconds without rounding.
- 'TimeRange' accepts decimal MJD as doubles or datetimes.

4.3 Work with quality flags

Flags are bit strings. Decode them with the meanings from the file's own header, then filter as coarsely or as finely as you need.

```
[t, obs, aux] = gracefoReading.readL1b(f);
[bits, report] = gracefoReading.decodeQualFlags(aux);
disp(report)            % bit, meaning, countSet, fracSet

% drop everything flagged
[t, obs] = gracefoReading.readL1b(f, 'ApplyQualityFlags', true);

% or only the bits that matter for your application
[t, obs, aux] = gracefoReading.readL1b(f, 'QualityBitMask', [6 7]);
fprintf('%d record(s) removed\n', aux.numRecordsFlagged);
```

- Bit 0 is the rightmost character of the bit string.
- The two quality options are mutually exclusive by design; asking for both raises conflictingQualityOptions.
- Filtering removes records, which creates gaps; see 4.4.

4.4 Find gaps and process contiguous segments

Filtering and data outages both break continuity. Most downstream operations, from differentiation to filtering, are only valid within a contiguous segment.

```
[gaps, stats] = gracefoReading.findGaps(t);          % rate auto-detected
fprintf('%d gap(s), longest %.0f s\n', stats.numGaps, stats.longestGapSec);
disp(gaps)

for s = 1:size(stats.segments, 1)
    idx = stats.segments(s,1):stats.segments(s,2);
    processSegment(t(idx,:), obs(idx,:));
end
```

- Pass 'NominalRate' when the series is dominated by gaps, since the automatic estimate is the median difference.
- All three time formats give identical results.
- stats.segments always has at least one row.

4.5 Batch process an archive

Locate files by product and date, check coverage, then read them as one series.

```
[files, rep] = gracefoReading.findL1bFiles(archive, "ACT1B", ...
    'DateRange', [datetime(2020,1,1) datetime(2020,1,31)], ...
    'Satellite', "C");
fprintf('%d file(s), %d day(s) missing\n', numel(files), numel(rep.missingDates));

[t, acc, aux] = gracefoReading.readL1bMulti(files, ...
    'ApplyQualityFlags', true, 'CacheDir', 'l1bcache');
disp(aux.recordsPerFile');   % contribution of each file
```

- Check rep.missingDates before a long run; a silent hole in the archive is worse than a failed search.
- readL1bMulti warns when the concatenated time is not strictly increasing, which catches wrong order and duplicates.
- aux.recordsPerFile lets you map any record back to its file.

4.6 Speed up repeated reads with caching

Parsing ASCII is the expensive part. The cache stores the parsed result under a SHA-256 digest of the file content and toolbox version, so it is safe by construction: change a byte, or upgrade the toolbox, and the entry is no longer used.

```
tic; gracefoReading.readL1b(f, 'CacheDir', 'l1bcache'); toc   % parse + store
tic; [t, obs, aux] = gracefoReading.readL1b(f, 'CacheDir', 'l1bcache'); toc
disp(aux.cacheHit);   % true

% post-parse options still apply to a cached read
tns = gracefoReading.readL1b(f, 'CacheDir', 'l1bcache', 'TimeFormat', 'gpsns');
```

- One cache entry serves every TimeRange, TimeFormat and quality setting, because those are applied after loading.
- The digest doubles as a provenance fingerprint: equal keys mean equal parse inputs.
- Delete the directory to clear the cache; nothing else refers to it.

4.7 Exchange data with GROOPS

Export a series in the GROOPS ASCII instrument convention and read it back to verify the roundtrip.

```
gracefoReading.writeGroopsInstrument('act.txt', t, acc(:,1:3), 'TypeCode', 3);
[t2, d2, info] = gracefoReading.readGroopsInstrument('act.txt');
fprintf('max |dt| = %.3g s\n', ...
    max(abs((t2(:,1)-t(:,1))*86400 + t2(:,2)-t(:,2))));
```

- GROOPS stores time as decimal MJD, which resolves about 1.3 microseconds around MJD 58849; that is the roundtrip limit.
- Verify the version tag and type code against your GROOPS installation before production use.
- Output is LF-only on every platform, so files are byte-identical.

4.8 Read classic GRACE data (experimental)

Files from the original GRACE mission use key-value headers that do not declare their columns. The toolbox detects them and applies built-in layouts, warning every time.

```
% built-in layout (ACC1B, SCA1B, KBR1B, GNV1B)
[t, obs, aux] = gracefoReading.readL1b('ACC1B_2008-01-01_A_02.txt');
disp(aux.headerFormat);          % 'classic'
disp(aux.classicHeaderFields);   % parsed KEY : VALUE pairs

% any other classic product needs an explicit layout
[t, obs] = gracefoReading.readL1b(f, 'ColumnNames', ...
    ["gps_time","GRACE_id","myval","qualflg"]);
```

- The built-in layouts are taken from the handbook and are NOT yet validated against real classic files. Confirm before scientific use, or pass 'ColumnNames'.
- Classic qualflg may be decimal rather than a bit string; then decodeQualFlags does not apply.
- Suppress the warning per session with warning('off', 'gracefoReading:readL1b:classicExperimental') once verified.

5 Worked examples

5.1 Executable demo

examples/demoGracefoReading.m walks through reading, flag decoding, nanosecond time, gap analysis, caching, GROOPS roundtrip and archive search using the two small real files bundled with the toolbox. It runs anywhere the toolbox is installed and doubles as a smoke test.

```
run(fullfile(toolboxRoot, 'examples', 'demoGracefoReading.m'))
```

5.2 Daily accelerometer series for a month

A complete, realistic script: locate files, read with quality filtering and caching, verify continuity, and hand the result to processing.

```
archive = '/data/gracefo/L1B';
files = gracefoReading.findL1bFiles(archive, "ACT1B", ...
    'DateRange', [datetime(2020,1,1) datetime(2020,1,31)], ...
    'Satellite', "C");
[~, rep] = gracefoReading.findL1bFiles(archive, "ACT1B", 'Satellite', "C");
assert(isempty(rep.missingDates), 'archive incomplete');

[t, acc, aux] = gracefoReading.readL1bMulti(files, ...
    'ApplyQualityFlags', true, 'CacheDir', fullfile(tempdir,'l1b'));

[gaps, stats] = gracefoReading.findGaps(t, 'NominalRate', 1);
fprintf('%d records, %d gap(s), %d missing epoch(s)\n', ...
    size(t,1), stats.numGaps, stats.totalMissing);

ix = ismember(aux.obsNames, {'lin_accl_x','lin_accl_y','lin_accl_z'});
gracefoReading.writeGroopsInstrument('act_202001.txt', t, acc(:,ix));
```

5.3 Attitude quality control

```
[t, q, aux] = gracefoReading.readL1b('SCA1B_2020-01-01_C_04.txt');
[ok, stats] = gracefoReading.checkQuaternionNorm(q, aux);
fprintf('max |norm-1| = %.3g over %d records\n', stats.maxDeviation, stats.n);
if stats.numViolations > 0
    disp(t(~ok, :));      % epochs to investigate
end
```

5.4 Thruster activity around a manoeuvre

```
[tt, thr, auxT] = gracefoReading.readL1b('THR1B_2020-01-01_C_04.txt');
[bits, rep] = gracefoReading.decodeQualFlags(auxT);
activity = sum(thr, 2, 'omitnan');
plot(tt(:,2)/3600, activity, '.');
xlabel('hours of day'); ylabel('sum of thruster counters');
% THR1B is event-driven: gaps here are physical, not data loss
[gaps, stats] = gracefoReading.findGaps(tt);
```

6 API reference

Generated from the MATLAB help blocks. In MATLAB, the same text is available with help or doc, e.g. help gracefoReading.readL1b.

gracefoReading.readL1b

Read one GRACE-FO or classic GRACE Level-1B ASCII product file.

```
[time, obs, aux] = gracefoReading.readL1b(fileName)
[time, obs, aux] = gracefoReading.readL1b(fileName, Name, Value)
```

Reads any GRACE-FO Level-1B RL04 ASCII product (ACC1B/ACT1B, SCA1B, KBR1B, LRI1B, GNV1B, THR1B, MAS1B, ...). These products share one structure: a YAML header terminated by "# End of YAML header" whose "variables:" list declares every data column in order, followed by whitespace-delimited records. The reader is driven entirely by the file's own header, so no release-dependent column tables are hard-coded.

Handled automatically: - single (gps_time) and split (time_intg + time_frac) time tags with unit-aware fractional scaling, read from each file's own header rather than assumed per product family: microseconds in the real IHK1B/MAS1B/THR1B/TNK1B files, nanoseconds in MAG1B, - bit-string flag columns (qualflg, prod_flag) preserved as character bit patterns and decoded into logical bit matrices, - prod_flag-controlled variable-length records (e.g. MAS1B); absent values are returned as NaN, - gzip-compressed input (.gz), decompressed in memory, - strict row-integrity checking with exact line-number reports, - optional content-hashed caching of parsed results.

Classic GRACE (2002-2017) files, whose headers are "KEY : VALUE" lines terminated by "END OF HEADER", are detected automatically. Because classic headers do not declare their columns, built-in per-product layouts or the 'ColumnNames' option supply them; this path is EXPERIMENTAL and warns on every use (see ACCURACY AND LIMITATIONS).

Input arguments

Name	Type	Size	Description	Options and default
`fileName`	string | char	1x1	Path to a Level-1B ASCII product file, plain or gzip-compressed (.gz). (validation: mustBeFile)	–
`'TimeVariable'`	string	1x1	Override of the automatic time-tag detection: one variable name, or two joined by '+', e.g. "time_intg+time_frac".	Options: any variable name declared in the header **Default:** `"" (automatic detection)`
`'TimeRange'`	double | datetime	1x2	Inclusive epoch window. Doubles are interpreted as decimal MJD, datetimes on the GPS time scale.	**Default:** `[] (no restriction)`
`'TimeFormat'`	string	1x1	Representation of the TIME output.	Options: "mjd" (integer MJD + seconds of day), "datetime" (GPS time scale), "gpsns" (int64 GPS nanoseconds past 2000-01-01 12:00:00) **Default:** `"mjd"`
`'ApplyQualityFlags'`	logical	1x1	If true, every record whose qualflg has ANY bit set is removed; the number removed is reported in aux.numRecordsFlagged. Mutually exclusive with 'QualityBitMask'.	Options: true | false **Default:** `false`
`'QualityBitMask'`	double	1xQ	Selective alternative to 'ApplyQualityFlags': removes records where any of the LISTED qualflg bits is set. Bit numbering is 0-based, bit 0 = rightmost character of the bit string. Bits beyond the flag width are ignored.	Options: non-negative integers **Default:** `[] (off)`

Name	Type	Size	Description	Options and default
`'CacheDir'`	string	1x1	Directory for content-hashed .mat parse caches, created if it does not exist. The cache key is the SHA-256 digest of file content + toolbox version + 'TimeVariable' + 'ColumnNames'; TimeRange, TimeFormat and quality filtering are applied after loading, so one entry serves every combination of them.	**Default:** `"" (caching disabled)`
`'ColumnNames'`	string	1xC	Classic-format files only: explicit column names in file order, overriding the built-in layout. Ignored for YAML-header files, which declare their own.	**Default:** `[] (use the built-in layout table)`

Output arguments

Name	Type	Size	Description	Options and default
`time`	double | datetime | int64	[N x 2] | [N x 1]	Epochs according to 'TimeFormat': [N x 2] integer MJD and seconds of day (default), datetime on the GPS time scale, or int64 GPS nanoseconds. GRACE(-FO) time tags count continuous GPS seconds past 2000-01-01 12:00:00 with no leap seconds; the MJD split uses floor/mod arithmetic.	–
`obs`	double	[N x M]	Observation columns: numeric variables that are not time tags, not char-typed, not bit-string flags and not "_id" identifiers. Column j corresponds to aux.obsNames{j}; values absent because of prod_flag are NaN.	–
`aux`	struct	1x1	Metadata and non-observation columns. Fields: obsNames, variableNames, variableUnits, variableAttributes, timeVariable, timeSystem, epochDescription, epochMJD, numRecordsHeader, numRecordsActual, numRecordsFlagged, header, product, fileName, headerFormat ('yaml' | 'classic'), cacheHit (logical), classicHeaderFields (classic only), qualflg / qualflgBits, prod_flag / prodFlagBits, and one field per further auxiliary column. See the HTML reference page for the full table.	–

Accuracy and limitations

- Time in the default "mjd" format is a double pair; around MJD 58849 the seconds-of-day double resolves about 2e-11 s, so the L1B time tags are represented without loss at both quantizations in use: microsecond, and the nanosecond tags of MAG1B, which 2e-11 s still resolves about fifty times over. The "gpsns" format is exact integer arithmetic and is the recommended handoff to nanosecond pipelines, since a single double cannot hold 2020-era GPS nanoseconds (~6.3e17 > 2^53) without rounding.
- Values are converted with str2double, i.e. to the nearest double; the ASCII products carry fewer significant digits than a double, so no precision is lost relative to the file.
- Quality filtering removes whole records; it does not repair or interpolate. Gaps that result are reported by gracefoReading.findGaps, not by this function.
- Classic GRACE support is EXPERIMENTAL: the built-in column layouts (ACC1B, SCA1B, KBR1B, GNV1B) are taken from the GRACE Level-1B handbook and have NOT been validated against real classic files; classic qualflg may be decimal rather than an 8-character bit string. Every classic read warns accordingly. Verify against a real file, or pass 'ColumnNames', before scientific use.
- Binary Level-1B files are not supported (ASCII products only).
- Caching trusts the SHA-256 content digest; if no JVM is available the digest cannot be computed and the file is parsed normally after a cacheUnavailable warning.

Errors

Identifier	Condition
`gracefoReading:readL1b:badHeader`	no recognized header end marker
`gracefoReading:readL1b:binaryNotSupported`	binary-format file
`gracefoReading:readL1b:noVariables`	no variables declared

Identifier	Condition
`gracefoReading:readL1b:noTime`	no time tag identified
`gracefoReading:readL1b:timeVariableNotFound`	'TimeVariable' not in header
`gracefoReading:readL1b:unknownFracUnit`	unsupported time_frac unit
`gracefoReading:readL1b:columnMismatch`	row-integrity violation
`gracefoReading:readL1b:noData`	no data records
`gracefoReading:readL1b:conflictingQualityOptions`	both quality options given
`gracefoReading:readL1b:unsupportedClassicProduct`	classic file without built-in layout and no 'ColumnNames'

Warnings

Identifier	Condition
`gracefoReading:readL1b:recordCountMismatch`	header/actual count differ
`gracefoReading:readL1b:cacheUnavailable`	hashing failed; parsed fresh
`gracefoReading:readL1b:classicExperimental`	classic layout assumed

Examples

plain read of one daily accelerometer file

```
[t, acc, aux] = gracefoReading.readL1b('ACT1B_2020-01-01_C_04.txt');
plot(t(:,2)/3600, acc(:, strcmp(aux.obsNames,'lin_accl_x')));
```

exact int64 nanoseconds, cached, selective flag filtering

```
[tns, m, aux] = gracefoReading.readL1b('MAS1B_2020-01-01_C_04.txt', ...
   'TimeFormat', 'gpsns', 'QualityBitMask', [6 7], ...
   'CacheDir', 'l1bcache');
```

one hour of a gzipped KBR1B file as datetime

```
[t, kbr] = gracefoReading.readL1b('KBR1B_2020-01-01_Y_04.txt.gz', ...
   'TimeFormat', 'datetime', ...
   'TimeRange', [datetime(2020,1,1,0,0,0), datetime(2020,1,1,1,0,0)]);
```

See also: gracefoReading.readL1bMulti, gracefoReading.findGaps, gracefoReading.findL1bFiles, gracefoReading.decodeQualFlags, gracefoReading.checkQuaternionNorm, gracefoReading.writeGroopsInstrument, gracefoReading.readGroopsInstrument, gracefoReading.version

gracefoReading.readL1bMulti

Read and concatenate several GRACE-FO L1B ASCII files.

```
[time, obs, aux] = gracefoReading.readL1bMulti(fileNames)
[time, obs, aux] = gracefoReading.readL1bMulti(fileNames, Name, Value)
```

Calls gracefoReading.readL1b on each file (e.g. consecutive daily ACT1B files) and concatenates the results into one continuous record. All files must expose the same observation columns (aux.obsNames); a mismatch raises an error rather than silently joining incompatible data. Per-file provenance is preserved in aux, and the concatenated time series is checked for strict monotonicity, which catches wrong file order and duplicate days. All name-value options are forwarded unchanged to readL1b.

Input arguments

Name	Type	Size	Description	Options and default
`fileNames`	string | cellstr	1xK	Files in concatenation (typically chronological) order; K = number of files. (validation: mustBeNonempty)	–
`'TimeVariable'`	string	1x1	Forwarded to readL1b (time-tag override).	**Default:** `"" (automatic detection)`
`'TimeRange'`	double | datetime	1x2	Forwarded to readL1b; applied to every file.	**Default:** `[] (no restriction)`
`'TimeFormat'`	string	1x1	Forwarded to readL1b.	Options: "mjd" | "datetime" | "gpsns" **Default:** `"mjd"`
`'ApplyQualityFlags'`	logical	1x1	Forwarded to readL1b.	Options: true | false **Default:** `false`
`'QualityBitMask'`	double	1xQ	Forwarded to readL1b (0-based bit numbers).	**Default:** `[] (off)`
`'CacheDir'`	string	1x1	Forwarded to readL1b; caching is per file, so a partially cached series only parses the new files.	**Default:** `"" (caching disabled)`
`'ColumnNames'`	string	1xC	Forwarded to readL1b (classic-format files only).	**Default:** `[] (use the built-in layout table)`

Output arguments

Name	Type	Size	Description	Options and default
`time`	double | datetime | int64	[N x 2] | [N x 1]	Concatenated epochs in the requested format; N = total records kept across all files. N = 0 is a valid result (see ACCURACY AND LIMITATIONS); the column count is preserved, so an empty mjd result is 0x2 and an empty datetime/gpsns result is 0x1.	–
`obs`	double	[N x M]	Concatenated observation columns, ordered as aux.obsNames.	–
`aux`	struct	1x1	obsNames, variableNames, variableUnits, variableAttributes, timeVariable, timeSystem, epochDescription, epochMJD taken from the first file; fileNames [K x 1], products [K x 1], headers [K x 1], recordsPerFile [K x 1] (cumsum gives the split points), numRecordsHeader/Actual/Flagged [K x 1], plus per-record auxiliary fields that are present with consistent type in every file, concatenated in file order; inconsistent fields are dropped with warning fieldDropped.	–

Accuracy and limitations

- Accuracy is that of readL1b; concatenation itself is exact.
- Records are joined in the order given, not sorted. Overlapping or out-of-order files trigger the timeNotMonotonic warning but are still returned, so the caller can decide how to resolve the overlap.
- Duplicate epochs are not removed automatically.

- If 'TimeRange' or a quality option removes every record of every file, the result is empty rather than an error: time and obs are the same clean empties readL1b returns for one file, aux still carries fileNames, products, recordsPerFile (all zero) and the per-file record counts, and the outcome is reported by the noRecordsInRange warning. The monotonicity check is skipped, an empty series being trivially monotonic. Callers that require data should test isempty(time) or promote the warning with warning('error', 'gracefoReading:readL1bMulti:noRecordsInRange').
- Memory grows linearly with the number of files; a full year of 1 Hz ACT1B is roughly 3e7 records. For long series, read in blocks or use 'CacheDir' with per-day processing.

Errors

Identifier	Condition
`gracefoReading:readL1bMulti:inconsistentVariables`	observation columns differ between files

Warnings

Identifier	Condition
`gracefoReading:readL1bMulti:fieldDropped`	aux field not concatenable
`gracefoReading:readL1bMulti:timeNotMonotonic`	time not strictly increasing across files
`gracefoReading:readL1bMulti:noRecordsInRange`	filters removed every record of every file

Examples

five consecutive days

```
files = compose("ACT1B_2020-01-%02d_C_04.txt", 1:5);
[t, acc, aux] = gracefoReading.readL1bMulti(files);
disp(aux.recordsPerFile');
```

a month located by findL1bFiles, cached and flag-filtered

```
files = gracefoReading.findL1bFiles(archive, "ACT1B", ...
   'DateRange', [datetime(2020,1,1) datetime(2020,1,31)], ...
   'Satellite', "C");
[t, acc] = gracefoReading.readL1bMulti(files, ...
   'ApplyQualityFlags', true, 'CacheDir', 'l1bcache');
```

a range the files do not cover: empty, not an error

```
tr = [datetime(2019,12,31,0,0,0), datetime(2019,12,31,1,0,0)];
[t, acc, aux] = gracefoReading.readL1bMulti(files, 'TimeRange', tr);
if isempty(t)
   fprintf('%d file(s) read, no record in range\n', numel(aux.fileNames));
end
```

See also: gracefoReading.readL1b, gracefoReading.findL1bFiles, gracefoReading.findGaps

gracefoReading.decodeQualFlags

Decode a bit-string flag column using the file's own header.

```
[bits, report] = gracefoReading.decodeQualFlags(aux)
[bits, report] = gracefoReading.decodeQualFlags(aux, flagName)
```

Decodes a bit-string flag column of a previously read Level-1B product into a logical matrix and summarises how often each bit is set. The bit meanings are taken from the flag_meanings attribute parsed out of the product file's own header (aux.variableAttributes), so the decoding always matches the file actually read and no release-dependent bit tables are hard-coded. Bits without a documented meaning are still reported, marked as not documented.

Input arguments

Name	Type	Size	Description	Options and default
`aux`	struct	1x1	Third output of gracefoReading.readL1b or readL1bMulti. Must contain the raw flag field as char [N x B] and the variableAttributes struct.	–
`flagName`	string | char	1x1	Name of the flag variable to decode.	Options: 'qualflg', 'prod_flag', or any other bit-string column present in aux **Default:** `'qualflg'`. `Optional, positional`

Output arguments

Name	Type	Size	Description	Options and default
`bits`	logical	[N x B]	Column b corresponds to bit b-1; bit 0 is the rightmost character of the bit string, following the GRACE-FO convention.	–
`report`	table	[B x 4]	One row per bit, variables: `bit` double : bit number (0-based) `meaning` string : meaning from the header, or "(not documented)" `countSet` double : records with this bit set `fracSet` double : fraction of records, range 0 to 1	–

Accuracy and limitations

- Decoding is exact: it is a character comparison, not a numerical conversion.
- Meanings are only as complete as the header of the file at hand. Different releases document different bits, and some products document none, in which case every meaning reads "(not documented)" - this is a property of the data, not a failure of the function.
- The order in which flag_meanings entries map to bit numbers follows the file's declaration order, assumed to start at bit 0. Where a header lists fewer meanings than the flag has characters, the surplus bits stay undocumented.
- Applies to bit-string columns only. Classic GRACE products may store qualflg as a decimal number, which this function does not decode (see readL1b limitations).

Errors

Identifier	Condition
`gracefoReading:decodeQualFlags:flagNotFound`	the requested flag column is not present in aux
`gracefoReading:decodeQualFlags:notBitString`	the requested column is not a character bit string

Examples

quality flags of an SCA1B day

```
[t, q, aux] = gracefoReading.readL1b('SCA1B_2020-01-01_C_04.txt');
[bits, report] = gracefoReading.decodeQualFlags(aux);
disp(report)
```

which MAS1B columns are actually present (prod_flag)

```
[~, ~, auxM] = gracefoReading.readL1b('MAS1B_2020-01-01_C_04.txt');
[~, rep] = gracefoReading.decodeQualFlags(auxM, 'prod_flag');
disp(rep(rep.fracSet > 0, :))
```

See also: gracefoReading.readL1b, gracefoReading.readL1bMulti

gracefoReading.checkQuaternionNorm

Verify unit norm of SCA1B attitude quaternions.

```
[ok, stats] = gracefoReading.checkQuaternionNorm(obs, aux)
[ok, stats] = gracefoReading.checkQuaternionNorm(obs, aux, Name, Value)
```

Checks, record by record, that the star-camera attitude quaternion read from an SCA1B product has unit norm - a cheap and effective detector of parsing errors, column mix-ups and corrupt records before the attitude data enter any rotation. The four components are located by name (quatangle, quaticoeff, quatjcoeff, quatkcoeff) in aux.obsNames, so obs can be passed exactly as returned by readL1b. Rotation and coordinate transformations are deliberately out of scope for this toolbox; only the norm sanity check is provided.

Input arguments

Name	Type	Size	Description	Options and default
`obs`	double	[N x M]	Observation matrix from readL1b or readL1bMulti of an SCA1B product.	–
`aux`	struct	1x1	Matching aux output; aux.obsNames is used to locate the four quaternion columns.	–
`'Tolerance'`	double	1x1	Maximum allowed absolute deviation |norm(q) - 1| per record.	Options: any positive value; 1e-6 is loose enough for the ASCII rounding of the products and tight enough to expose real problems **Default:** `1e-6`

Output arguments

Name	Type	Size	Description	Options and default
`ok`	logical	[N x 1]	True where |norm(q) - 1| <= Tolerance.	–
`stats`	struct	1x1	n double : records checked `numViolations` double : count of ok == false `maxDeviation` double : max |norm(q) - 1| `rmsDeviation` double : rms of (norm(q) - 1) `tolerance` double : tolerance used `columns` double [1 x 4]: obs column indices in the order quatangle, i, j, k	–

Accuracy and limitations

- The ASCII products store quaternion components with finite digits, so a small nonzero deviation is normal; real SCA1B files typically show |norm-1| of order 1e-9 to 1e-8.
- Unit norm is necessary but not sufficient for correctness: a quaternion can be normalised and still describe a wrong attitude, and a sign flip (q and -q describe the same rotation) is invisible to this test.
- The check requires all four components in obs; if a product stores any of them as an auxiliary column the function raises columnsNotFound rather than guessing.
- No renormalisation is performed. Correcting attitude data is left to the processing that uses it.

Errors

Identifier	Condition
`gracefoReading:checkQuaternionNorm:columnsNotFound`	one of the four quaternion columns is absent from aux.obsNames, i.e. obs is not from an SCA1B-like product

Examples

quick verdict for one day

```
[t, q, aux] = gracefoReading.readL1b('SCA1B_2020-01-01_C_04.txt');
[ok, stats] = gracefoReading.checkQuaternionNorm(q, aux);
fprintf('max |norm-1| = %.3g, violations: %d\n', ...
       stats.maxDeviation, stats.numViolations);
```

inspect the offending epochs

```
bad = find(~ok);
disp(t(bad, :));
```

See also: gracefoReading.readL1b, gracefoReading.decodeQualFlags

gracefoReading.writeGroopsInstrument

Export time and observations as a GROOPS ASCII instrument file.

```
gracefoReading.writeGroopsInstrument(fileName, time, data)
gracefoReading.writeGroopsInstrument(fileName, time, data, Name, Value)
```

Writes an ASCII instrument file in the GROOPS convention

groops instrument version=<version> <typeCode> <dataColumns> (type/dimension header) <numRecords> (record-count line) <decimalMJD> <data1> <data2> ... (one line per record)

closing the loop from PO.DAAC Level-1B files read with this toolbox to GROOPS-based processing without intermediate scripts. Output always uses Unix line endings (LF) on every platform, so files written on Windows and Linux are byte-identical.

Input arguments

Name	Type	Size	Description	Options and default
`fileName`	string | char	1x1	Output path; an existing file is overwritten.	–
`time`	double | datetime	[N x 2] | [N x 1]	Epochs as returned by gracefoReading.readL1b: integer MJD and seconds of day, or datetime on the GPS time scale. Converted to the decimal MJD that GROOPS expects.	–
`data`	double	[N x M]	Observation columns to write; M may be zero, which produces a time-only file.	–
`'TypeCode'`	double	1x1	GROOPS epoch-type code written as the first integer of the type/dimension header.	Options: integer-valued; 0 denotes a generic MISCVALUES-style record, other codes denote typed records (accelerometer, star camera, ...) as defined by the GROOPS release in use **Default:** `0`
`'Version'`	double	1x1	Version tag written on the first line.	Options: integer-valued, e.g. 20200123 **Default:** `20200123`
`'Format'`	string	1x1	printf format applied to each data value.	Options: any valid floating-point conversion, e.g. "%.16e" for full double precision or "%.6f" for compact output **Default:** `"%.16e"`

Output arguments

None; the function works by side effect.

Accuracy and limitations

- Time is written as decimal MJD in a single double, as the GROOPS convention requires. Around MJD 58849 a double resolves about 1.3 microseconds, which is adequate for 1 Hz Level-1B products but not for sub-microsecond timing; such applications must not round-trip through decimal MJD. The int64 "gpsns" format of readL1b preserves the full time tag inside MATLAB.
- Data values are written with 'Format'; the default keeps full double precision, a shorter format silently truncates.
- The header conventions (version tag, type codes) reflect the documented GROOPS ASCII layout but have NOT been verified against a file produced by a specific GROOPS installation; confirm both before relying on this export in production (see HANDOVER.md, open topics). gracefoReading.readGroopsInstrument reads the format back and makes that check a two-line test.
- NaN values are written as the platform's printf representation of NaN; GROOPS may not accept them, so filter or fill gaps before export.

Errors

Identifier	Condition
`gracefoReading:writeGroopsInstrument:sizeMismatch`	number of rows of time and data differ
`gracefoReading:writeGroopsInstrument:openFailed`	the output file could not be opened for writing

Examples

export the three linear accelerations of one day

```
[t, acc, aux] = gracefoReading.readL1b('ACT1B_2020-01-01_C_04.txt');
gracefoReading.writeGroopsInstrument('act.txt', t, acc(:,1:3));
```

typed record with compact formatting, then verify

```
gracefoReading.writeGroopsInstrument('act.txt', t, acc(:,1:3), ...
   'TypeCode', 3, 'Format', "%.9e");
[t2, d2, info] = gracefoReading.readGroopsInstrument('act.txt');
```

See also: gracefoReading.readGroopsInstrument, gracefoReading.readL1b

gracefoReading.version

Version and release metadata of the gracefoReading toolbox.

```
v = gracefoReading.version()
[v, info] = gracefoReading.version()
```

Returns the toolbox version as a semantic version string, allowing downstream scripts and processing chains to assert a minimum version programmatically instead of trusting folder names. The second output adds release metadata. The version string is also part of the cache key used by readL1b, so caches written by an older toolbox are never reused after an upgrade.

Input arguments

None.

Output arguments

Name	Type	Size	Description	Options and default
`versionString`	char	[1 x C]	Semantic version, e.g. '1.0.2'.	–
`info`	struct	1x1	`version` char : same as versionString `date` char : release date 'yyyy-MM-dd' `developedWith` char : authorship and model note `license` char : 'MIT'	–

Accuracy and limitations

- The version is a literal maintained in this file and mirrored in Contents.m and docsrc/functionSpecs.json; the consistency test testDocConsistency enforces that the three agree. Tests must never pin a version literal themselves, since that breaks on the next bump.
- String comparison of semantic versions is lexicographic and only reliable while all components stay single-digit; for robust checks compare the numeric parts, e.g. sscanf(gracefoReading.version(), '%d.%d.%d').

Examples

simple assertion in a processing script

```
assert(string(gracefoReading.version()) >= "1.0.0")
```

numeric comparison, robust beyond version 9

```
% (this example asserted v >= 2.3 until 2026-09-08; the renumbering
%  from 2.3.0 to 1.0.0 left it impossible to satisfy, which is what
%  a runnable EXAMPLE section exists to expose)
v = sscanf(gracefoReading.version(), '%d.%d.%d')';
assert(v(1) > 1 || (v(1) == 1 && v(2) >= 0))
```

See also: gracefoReading.readL1b

gracefoReading.findGaps

Detect sampling gaps and contiguous segments in an L1B time series.

```
[gaps, stats] = gracefoReading.findGaps(time)
[gaps, stats] = gracefoReading.findGaps(time, Name, Value)
```

Analyses the epoch spacing of a time series read with gracefoReading.readL1b or readL1bMulti and reports every gap exceeding Tolerance times the nominal sampling interval, together with the contiguous segments between the gaps. Downstream processing - filtering, arc segmentation, differentiation - requires contiguous data, so this moves a routine and error-prone preprocessing step into tested code. All three readL1b time formats are accepted and produce identical results; gap epochs are always reported as MJD and seconds of day.

Input arguments

Name	Type	Size	Description	Options and default
`time`	double | datetime | int64	[N x 2] | [N x 1]	Epochs as returned by readL1b: [N x 2] integer MJD and seconds of day, datetime on the GPS time scale, or int64 GPS nanoseconds past 2000-01-01 12:00:00. Must be sorted ascending.	–
`'NominalRate'`	double	1x1	Nominal sampling interval in seconds (e.g. 1 for ACT1B, 5 for KBR1B, 2 for LRI1B, 3600 for MAS1B).	Options: any positive value, or NaN to estimate the rate as the median of the epoch differences **Default:** `NaN (estimate from the data)`
`'Tolerance'`	double	1x1	A step dt counts as a gap when dt > Tolerance * NominalRate.	Options: any positive value; 1.5 tolerates jitter of half a sampling interval **Default:** `1.5`

Output arguments

Name	Type	Size	Description	Options and default
`gaps`	table	[G x 8]	One row per detected gap, variables: `indexBefore` double : record index before the gap `indexAfter` double : record index after the gap `mjdBefore` double : integer MJD of last record `secBefore` double : seconds of day of last record `mjdAfter` double : integer MJD of next record `secAfter` double : seconds of day of next record `durationSec` double : gap length in seconds `missingRecords` double : round(durationSec/rate) - 1 `Empty` table : (G = 0) if the series is contiguous.	–
`stats`	struct	1x1	 n double : epochs analysed `nominalRate` double : sampling interval used [s] `rateEstimated` logical : true if estimated from data `numGaps` double : height(gaps) `totalMissing` double : sum of missingRecords `longestGapSec` double : 0 if no gaps `segments` double [G+1 x 2]: first and last record index of each contiguous segment	–

Accuracy and limitations

- Differences are formed relative to the first epoch in double precision. Over a one-day series the resolution is better than 1e-9 s, far below any physically meaningful gap, so the double reduction is harmless even for int64 input.
- Automatic rate estimation uses the median difference and therefore requires that most steps are nominal; for series dominated by gaps, pass 'NominalRate' explicitly.
- missingRecords is a count derived from the assumed constant rate, rounded and floored at 1; it is an estimate, not a statement about which epochs the data centre actually intended to deliver.

- Duplicate epochs (dt = 0) are not gaps and are not reported here; use the monotonicity warning of readL1bMulti.
- The input must be sorted; unsorted input raises an error rather than being silently sorted, since sorting would break the correspondence to obs.

Errors

Identifier	Condition
`gracefoReading:findGaps:notSorted`	time not ascending
`gracefoReading:findGaps:badTimeFormat`	unrecognized time layout

Examples

automatic rate detection on a KBR1B day (5 s nominal)

```
[t, kbr] = gracefoReading.readL1b('KBR1B_2020-01-01_Y_04.txt');
[gaps, stats] = gracefoReading.findGaps(t);
fprintf('%d gap(s), %d record(s) missing\n', stats.numGaps, stats.totalMissing);
```

process each contiguous segment separately

```
[~, stats] = gracefoReading.findGaps(t, 'NominalRate', 5);
for s = 1:size(stats.segments, 1)
   idx = stats.segments(s,1):stats.segments(s,2);
   processSegment(t(idx,:), kbr(idx,:));
end
```

See also: gracefoReading.readL1b, gracefoReading.readL1bMulti

gracefoReading.findL1bFiles

Locate GRACE-FO L1B files in an archive by product and date.

```
files = gracefoReading.findL1bFiles(archiveRoot, product)
[files, report] = gracefoReading.findL1bFiles(archiveRoot, product, Name, Value)
```

Searches an archive folder for PO.DAAC-named product files, for example ACT1B_2020-01-03_C_04.txt or the gzipped variant ACT1B_2020-01-03_C_04.txt.gz, sorts them chronologically and reports date coverage including calendar days without a file. Together with readL1bMulti this reduces multi-month batch reading to two lines. Files whose names do not match the PO.DAAC pattern are ignored, and an empty result is returned rather than raised as an error, so the function can be used to probe an archive.

Input arguments

Name	Type	Size	Description	Options and default
`archiveRoot`	string | char	1x1	Folder to search. (validation: mustBeFolder)	–
`product`	string	1x1	Product identifier as it appears at the start of the file name.	Options: "ACT1B", "ACC1B", "SCA1B", "KBR1B", "LRI1B", "GNV1B", "THR1B", "MAS1B", ... Required, positional
`'DateRange'`	datetime	1x2	Inclusive date window; order of the two elements is irrelevant. Also defines the span over which missing days are reported.	**Default:** `[] (all dates found)`
`'Satellite'`	string	1x1	Satellite letter in the file name.	Options: "C" | "D" (GRACE-FO 1 and 2), "Y" (dual- satellite products such as KBR1B/LRI1B), "" for any **Default:** `"" (any satellite)`
`'Release'`	string	1x1	Release tag in the file name.	Options: "04", "03", **Default:** `"04"`
`'Recursive'`	logical	1x1	Search subfolders as well, which suits archives organised by year or month.	Options: true | false **Default:** `true`

Output arguments

Name	Type	Size	Description	Options and default
`files`	string	[K x 1]	Full paths sorted by date and then satellite; empty string array if nothing matches.	–
`report`	struct	1x1	`dates` datetime [K x 1]: date of each file `satellites` string [K x 1]: satellite letters `firstDate` datetime : NaT if no files `lastDate` datetime : NaT if no files `missingDates` datetime [D x 1]: calendar days with no file inside [firstDate, lastDate], or inside 'DateRange' when it is given	–

Accuracy and limitations

- Dates come from the file NAME, not from the file content; a misnamed file is placed wrongly. Content dates are available from readL1b via the time output.
- Only the PO.DAAC naming convention PROD1B_yyyy-MM-dd_S_RR.txt[.gz] is recognized. Archives using other conventions need their own listing; the resulting string array can be passed to readL1bMulti directly.
- Missing-day detection assumes daily files; it is not meaningful for products delivered at another cadence.
- Recursive search visits every subfolder of archiveRoot and can be slow on network file systems with very large archives; set 'Recursive' to false when the layout is flat.
- Duplicates of the same date in different subfolders are all returned; the caller decides which to keep.

Examples

one month of ACT1B for GRACE-FO 1, then read it

```
files = gracefoReading.findL1bFiles(archive, "ACT1B", ...
   'DateRange', [datetime(2020,1,1) datetime(2020,1,31)], ...
   'Satellite', "C");
[t, acc, aux] = gracefoReading.readL1bMulti(files);
```

check archive completeness before a long run

```
[~, rep] = gracefoReading.findL1bFiles(archive, "KBR1B", 'Satellite', "Y");
fprintf('%s .. %s, %d day(s) missing\n', rep.firstDate, rep.lastDate, ...
       numel(rep.missingDates));
```

See also: gracefoReading.readL1bMulti, gracefoReading.readL1b, gracefoReading.findGaps

gracefoReading.readGroopsInstrument

Read a GROOPS ASCII instrument file.

```
[time, data, info] = gracefoReading.readGroopsInstrument(fileName)
[time, data, info] = gracefoReading.readGroopsInstrument(fileName, ...
Name, Value)
```

Reads an ASCII instrument file in the convention written by gracefoReading.writeGroopsInstrument: a version line, a two-integer type and dimension header, a lone record count and one record per line starting with the decimal MJD. It is the counterpart of the writer and serves two purposes: roundtrip verification of the export, and reading GROOPS-produced series back into MATLAB for comparison with Level-1B data.

The decimal-MJD time tag is recovered from the DIGITS of the record line, not from the double that sscanf would return. The integer and fractional parts are split at the decimal point and converted with int64 arithmetic, so the epoch is exact to every digit the file prints. Reading the same token through a double loses the low digits: a double holds MJD 58849 to an ulp of 0.63 microseconds, and over 200 000 epochs of 2020-01-01 the double path misplaced them by up to 314 ns while the digit path was exact. That is why 'TimeFormat', "gpsns" is offered here at all - it would otherwise return nine digits of which three are noise.

Input arguments

Name	Type	Size	Description	Options and default
`fileName`	string | char	1x1	Path to a GROOPS ASCII instrument file. (validation: mustBeFile)	–
`'TimeFormat'`	string	1x1	Representation of the TIME output, matching gracefoReading.readL1b so the two readers hand off to the same downstream code.	Options: "mjd" (integer MJD + seconds of day), "datetime" (GPS time scale), "gpsns" (int64 GPS nanoseconds past 2000-01-01 12:00:00) **Default:** `"mjd"`

Output arguments

Name	Type	Size	Description	Options and default
`time`	double | datetime | int64	[N x 2] | [N x 1]	Epochs according to 'TimeFormat': integer MJD and seconds of day (default), datetime on the GPS time scale, or int64 GPS nanoseconds. The MJD column and the nanosecond count are both derived by integer arithmetic from the printed digits.	–
`data`	double	[N x M]	Data columns; M equals info.dataColumns.	–
`info`	struct	1x1	`version` double : version tag from the first line `typeCode` double : first integer of the header `dataColumns` double : declared number of data columns `numRecords` double : declared number of records	–

Accuracy and limitations

- Time is exact with respect to what the file prints. It is NOT exact with respect to a series that went through the writer: writeGroopsInstrument forms the decimal MJD as a double before printing it, so a write-then-read roundtrip is limited to about 0.6 microseconds by the writer, not by this reader. Only a file whose digits carry true sub-microsecond information delivers it.
- At most the first 15 fractional digits are used. The 15th digit of a day is 86 picoseconds, so this cannot affect a nanosecond result; digits beyond it are discarded rather than rounded.
- Rounding the fractional digits to whole nanoseconds can reach a full day (a tag within about 43 ps of midnight). The carry into the MJD column is applied, so seconds of day stays in [0, 86400).
- Time tags must be plain fixed-point decimals. An exponential tag such as 5.8849e4 raises badTimeToken rather than being read through a double, because the digit path is what makes the nanosecond output meaningful. A sign is accepted only as '+': MJD is positive for every real epoch, and a negative tag would need floor semantics the integer split does not implement.

- Time tags are validated before the declared column count, so a malformed tag raises badTimeToken and not countMismatch. The other order names the wrong cause: a tag like "58849.5.5" makes sscanf return three values where two columns were declared, and the reader would report a count error for a time problem.
- Only the ASCII instrument format is read; GROOPS binary files are not supported.
- Declared record and column counts are enforced, which catches truncated files, but the type code is not interpreted: the function does not know what the columns mean. Consult the GROOPS documentation of the type code in info.
- Header conventions have not been verified against a specific GROOPS installation (see writeGroopsInstrument limitations); files from a real installation may carry additional comment lines, which this reader does not expect. It is therefore NOT established that this function reads the TU Graz operational orbit products - see obligation O-8 in HANDOVER.md.

Errors

Identifier	Condition
`gracefoReading:readGroopsInstrument:badFormat`	first line or header lines not in the expected convention
`gracefoReading:readGroopsInstrument:countMismatch`	declared record or column count does not match the data
`gracefoReading:readGroopsInstrument:badTimeToken`	a time tag is not a plain fixed-point decimal MJD

Examples

roundtrip check of the exporter

```
gracefoReading.writeGroopsInstrument('acc.txt', t, acc);
[t2, acc2, info] = gracefoReading.readGroopsInstrument('acc.txt');
fprintf('max |dt| = %.3g s\n', ...
   max(abs((t2(:,1)-t(:,1))*86400 + t2(:,2)-t(:,2))));
```

compare a GROOPS-processed series with Level-1B data

```
[tg, dg] = gracefoReading.readGroopsInstrument('fromGroops.txt');
[tl, obs] = gracefoReading.readL1b('ACT1B_2020-01-01_C_04.txt');
```

exact int64 nanoseconds, for a lossless handoff

```
[tns, dg] = gracefoReading.readGroopsInstrument('fromGroops.txt', ...
   'TimeFormat', 'gpsns');
```

See also: gracefoReading.writeGroopsInstrument, gracefoReading.readL1b

7 Accuracy, limitations and known issues

7.1 Numerical accuracy

- Values are converted with str2double, i.e. to the nearest double. The ASCII products carry fewer significant digits than a double, so nothing is lost relative to the file.
- The default time format is a double pair (integer MJD, seconds of day) and resolves better than 1e-10 s within a day.
- The "gpsns" format is exact integer arithmetic and is the recommended handoff to nanosecond pipelines; a single double cannot represent 2020-era GPS nanoseconds exactly.
- GROOPS export uses decimal MJD, limiting exchange with GROOPS to about 1.3 microseconds around MJD 58849.

7.2 Known limitations

- Binary Level-1B products are not supported; ASCII only.
- Classic GRACE (2002-2017) support is experimental: the built-in column layouts come from the handbook and are not yet validated against real classic files. Every classic read warns.
- GROOPS header conventions (version tag, type codes) have not been verified against a specific GROOPS installation.
- findL1bFiles recognizes the PO.DAAC naming convention only, and takes dates from file names rather than content.
- readL1bMulti concatenates in the given order without sorting or de-duplicating; it warns rather than repairing.
- Caching and gzip decompression use the JVM where available and fall back safely under -nojvm, with a warning in the cache case.

7.3 Verified against real data

The acceptance suite reads genuine PO.DAAC RL04 files of 2020-01-01 for all seven products and compares record counts, first and last epochs, observation counts, flag handling and the MAS1B variable-length layout against an independent Python reference implementation. This exists because version 1.0 of this toolbox was written against invented header conventions and failed on every real file; synthetic fixtures alone cannot catch that class of error.

7.4 Version history

Version	Date	Highlights
`1.0.2`	2026-09-09	Fixed — MAG1B was unreadable; Fixed — two public help claims that had become wrong; Changed — version; Notes
`1.0.1`	2026-09-08	Fixed — readL1bMulti crashed where readL1b returned empties; Fixed — an EXAMPLE that could not run; Changed — version; Known deviations from VERSIONING.md (not addressed here)
`1.0.0`	2026-09-06	Changed — version scheme; Fixed — the toolbox could not be loaded at all; Fixed — the cause, not just the symptom; Added — checks that do not share that assumption; Known, not addressed here
`2.3.0`	2026-08-14	Added — CI and build; Added — documentation; Changed
`2.2.0`	2026-08-01	Added — pipeline integration; Added — batch capability; Added — strategic; Added — engineering; Changed (internal)
`2.1.1`	2026-08-01	Fixed (Windows)
`2.1.0`	2026-08-01	Changed (layout / installation)
`2.0.2`	2026-08-01	Fixed; Release process

The complete change log is CHANGELOG.md in the toolbox root.